Building a Python Netcat Clone

A Deep Dive into TCP Sockets, Threading, and Network Programming

Based on *Black Hat Python* by Justin Seitz — Chapter 2

1 Introduction

Netcat is one of the most fundamental tools in a penetration tester's toolkit. It creates raw TCP/UDP connections, spawns shells, transfers files, and much more. But instead of just using it, *Black Hat Python* challenges you to rebuild it from scratch in Python — so you truly understand what happens under the hood every time you run `nc -lvp 4444`.

This document walks through the complete implementation: every import, every class, every method — explained chunk by chunk in plain language, followed by the final production-ready code and a testing guide.

> **Goal:** When you finish this, running netcat in a real engagement should feel transparent — you know exactly what that TCP handshake, that shell spawn, and that file transfer look like at the socket level.

2 What is Netcat?

Netcat (`nc`) is a Unix utility that reads and writes data across network connections using TCP or UDP. In real engagements, penetration testers use the actual binary. The Python reimplementation exists purely for educational purposes — to demystify what the tool does internally.

- Open a TCP listener on any port
- Connect to a remote TCP endpoint
- Pipe data in and out of those connections
- Spawn an interactive shell over a network connection
- Transfer files between machines

3 Prerequisites

- Python 3.x installed
- Basic understanding of Python functions and classes
- A terminal (two terminals for testing)
- Optionally: two VMs (Kali + Metasploitable) for cross-machine testing

4 Code Walkthrough — Chunk by Chunk

The implementation is broken into eight logical chunks. Each chunk is presented with its code, followed by a plain-language explanation.

4.1 Chunk 1 — Imports

```python
import argparse
import socket
import shlex
import subprocess
import sys
import textwrap
import threading
```

All standard library modules — nothing needs to be installed.

accent accent	**Network layer** Provides the low-level TCP/IP socket API. Every connection, every byte sent and received, goes through this module.
accent accent	**Command execution** Runs shell commands like `ls`, `whoami`, `id` and captures their output.
accent accent	**Concurrency** Allows the server to handle multiple clients simultaneously. Each connection gets its own thread.
accent accent	**CLI interface** Parses command-line flags like `-t`, `-p`, `-l`, `-c`.
accent accent	**Safe splitting** Splits a command string like `"cat /etc/passwd"` into `["cat", "/etc/passwd"]` safely.

4.2 Chunk 2 — The execute() Helper

```
def execute(cmd):
    cmd = cmd.strip()
    if not cmd:
        return
    output = subprocess.check_output(shlex.split(cmd),
                                     stderr=subprocess.STDOUT)
    return output.decode()
```

A standalone helper. Give it a command string like `"whoami"` and it runs it on the local system and returns the output as a plain string.

- `cmd.strip()` — removes leading/trailing whitespace and newlines.
- `subprocess.check_output()` — runs the command and captures output.
- `stderr=subprocess.STDOUT` — merges error output into stdout so both get sent back.
- `.decode()` — converts raw bytes into a Python string.

4.3 Chunk 3 — NetCat.__init__()

```
class NetCat:
    def __init__(self, args, buffer=None):
        self.args = args
        self.buffer = buffer
        self.socket = socket.socket(socket.AF_INET,
            socket.SOCK_STREAM)
        self.socket.setsockopt(socket.SOL_SOCKET,
            socket.SO_REUSEADDR, 1)
```

The constructor runs when you create a `NetCat` object. It sets up everything before any connection is made.

- `socket.AF_INET` — specifies IPv4 addressing.
- `socket.SOCK_STREAM` — specifies TCP (vs. UDP which would be `SOCK_DGRAM`).
- `SO_REUSEADDR` — allows the port to be reused immediately after closing. Without this, you get *"Address already in use"* errors when restarting quickly.

4.4 Chunk 4 — run()

```
    def run(self):
        if self.args.listen:
            self.listen()
        else:
            self.send()
```

A simple router. If `-l` was passed, become a server. Otherwise, become a client.

> **Design note:** Separating `run()` from `listen()` and `send()` keeps the routing logic clean and makes each mode independently testable.

4.5 Chunk 5 — send() (Client Mode)

```
    def send(self):
        self.socket.connect((self.args.target, self.args.port))
        if self.buffer:
            self.socket.send(self.buffer)
        try:
            while True:
                recv_len = 1
                response = ''
                while recv_len:
                    data = self.socket.recv(4096)
                    recv_len = len(data)
                    response += data.decode()
                    if recv_len < 4096:
                        break
                if response:
                    print(response)
                    buffer = input('> ')
                    buffer += '\n'
                    self.socket.send(buffer.encode())
        except KeyboardInterrupt:
            print('User terminated.')
            self.socket.close()
            sys.exit()
```

- `connect()` — performs the TCP three-way handshake to the target.
- `recv(4096)` — receives up to 4096 bytes. The inner loop keeps receiving until the chunk is smaller than 4096, signalling no more data is immediately available.
- `input('> ')` — blocks and waits for you to type a command, then sends it.
- `buffer += '\n'` — appends a newline. The server uses `\n` as the signal that a complete command has arrived.
- `KeyboardInterrupt` — Ctrl+C cleanly closes the socket and exits.

4.6 Chunk 6 — listen() (Server Mode)

```
    def listen(self):
        self.socket.bind((self.args.target, self.args.port))
        self.socket.listen(5)
        while True:
            client_socket, _ = self.socket.accept()
            client_thread = threading.Thread(
                target=self.handle, args=(client_socket,)
            )
            client_thread.start()
```

- `bind()` — claims the IP address and port.

- `listen(5)` — starts listening. Argument 5 is the backlog — pending connections to queue.
- `accept()` — blocks until a client connects. Returns a new socket for that client.
- `threading.Thread()` — spawns a thread for each client so multiple can connect simultaneously.

4.7 Chunk 7 — handle() (Per-Client Logic)

```
    def handle(self, client_socket):
        if self.args.execute:
            output = execute(self.args.execute)
            client_socket.send(output.encode())

        elif self.args.upload:
            file_buffer = b''
            while True:
                data = client_socket.recv(4096)
                if data:
                    file_buffer += data
                else:
                    break
            with open(self.args.upload, 'wb') as f:
                f.write(file_buffer)
            message = f'Saved file {self.args.upload}'
            client_socket.send(message.encode())

        elif self.args.command:
            cmd_buffer = b''
            while True:
                try:
                    client_socket.send(b'BHP: #> ')
                    while '\n' not in cmd_buffer.decode():
                        cmd_buffer += client_socket.recv(64)
                    response = execute(cmd_buffer.decode())
                    if response:
                        client_socket.send(response.encode())
                    cmd_buffer = b''
                except Exception as e:
                    print(f'server killed {e}')
                    self.socket.close()
                    sys.exit()
```

Three distinct modes:

4.7.1 Execute mode (`-e "command"`)

Runs one command immediately when a client connects, sends the output back, and finishes. No interaction loop. Used for one-shot remote command execution.

4.7.2 Upload mode (`-u filename`)

Receives bytes from the client in a loop until the connection closes, then saves all of it to the specified file.

4.7.3 Command shell mode (-c)

The interactive mode. Sends a prompt (`BHP: #>`) to the client, waits for a complete command terminated by a newline, runs it via `execute()`, sends the output back, and loops forever.

4.8 Chunk 8 — main block

```
if __name__ == '__main__':
    parser = argparse.ArgumentParser(...)
    # ... add_argument calls ...
    args = parser.parse_args()

    if args.listen:
        buffer = ''
    else:
        buffer = sys.stdin.read()

    nc = NetCat(args, buffer.encode())
    nc.run()
```

- `sys.stdin.read()` — reads from standard input. This is how piping works: `echo 'hello' | python3 netcat.py ...` sends data via stdin.
- `buffer = ''` — when listening, don't read stdin (it would block forever).

5 Final Production Code

> The `NetCat` class is defined before the main block, and all indentation has been fixed from the book's printed version.

```
import argparse
import socket
import shlex
import subprocess
import sys
import textwrap
import threading

def execute(cmd):
    cmd = cmd.strip()
    if not cmd:
        return
    output = subprocess.check_output(shlex.split(cmd),
                                     stderr=subprocess.STDOUT)
    return output.decode()

class NetCat:
    def __init__(self, args, buffer=None):
        self.args = args
        self.buffer = buffer
```

```
        self.socket = socket.socket(socket.AF_INET,
            socket.SOCK_STREAM)
        self.socket.setsockopt(socket.SOL_SOCKET,
            socket.SO_REUSEADDR, 1)

    def run(self):
        if self.args.listen:
            self.listen()
        else:
            self.send()

    def send(self):
        self.socket.connect((self.args.target, self.args.port))
        if self.buffer:
            self.socket.send(self.buffer)
        try:
            while True:
                recv_len = 1
                response = ''
                while recv_len:
                    data = self.socket.recv(4096)
                    recv_len = len(data)
                    response += data.decode()
                    if recv_len < 4096:
                        break
                if response:
                    print(response)
                    buffer = input('> ')
                    buffer += '\n'
                    self.socket.send(buffer.encode())
        except KeyboardInterrupt:
            print('User terminated.')
            self.socket.close()
            sys.exit()

    def listen(self):
        self.socket.bind((self.args.target, self.args.port))
        self.socket.listen(5)
        while True:
            client_socket, _ = self.socket.accept()
            client_thread = threading.Thread(
                target=self.handle, args=(client_socket,)
            )
            client_thread.start()

    def handle(self, client_socket):
        if self.args.execute:
            output = execute(self.args.execute)
            client_socket.send(output.encode())
        elif self.args.upload:
            file_buffer = b''
            while True:
                data = client_socket.recv(4096)
                if data:
                    file_buffer += data
                else:
                    break
            with open(self.args.upload, 'wb') as f:
```

```
                f.write(file_buffer)
            message = f'Saved file {self.args.upload}'
            client_socket.send(message.encode())
        elif self.args.command:
            cmd_buffer = b''
            while True:
                try:
                    client_socket.send(b'BHP: #> ')
                    while '\n' not in cmd_buffer.decode():
                        cmd_buffer += client_socket.recv(64)
                    response = execute(cmd_buffer.decode())
                    if response:
                        client_socket.send(response.encode())
                    cmd_buffer = b''
                except Exception as e:
                    print(f'server killed {e}')
                    self.socket.close()
                    sys.exit()

if __name__ == '__main__':
    parser = argparse.ArgumentParser(
        description='BHP Net Tool',
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog=textwrap.dedent('''Example:
    netcat.py -t 192.168.1.108 -p 5555 -l -c # command shell
    netcat.py -t 192.168.1.108 -p 5555 -l -u=mytest.txt # upload
    netcat.py -t 192.168.1.108 -p 5555 -l -e="cat /etc/passwd"
    echo 'ABC' | ./netcat.py -t 192.168.1.108 -p 135
    netcat.py -t 192.168.1.108 -p 5555
    '''))
    parser.add_argument('-c', '--command', action='store_true')
    parser.add_argument('-e', '--execute')
    parser.add_argument('-l', '--listen', action='store_true')
    parser.add_argument('-p', '--port', type=int, default=5555)
    parser.add_argument('-t', '--target', default='192.168.1.203')
    parser.add_argument('-u', '--upload')
    args = parser.parse_args()
    if args.listen:
        buffer = ''
    else:
        buffer = sys.stdin.read()
    nc = NetCat(args, buffer.encode())
    nc.run()
```

6 Testing Guide

All tests use `127.0.0.1` (localhost). For cross-VM testing, replace with the target machine's IP.

6.1 Test 1 — Interactive Command Shell

```
# Terminal 1 (server)
```

```
python3 netcat.py -t 127.0.0.1 -p 5555 -l -c

# Terminal 2 (client)
python3 netcat.py -t 127.0.0.1 -p 5555
```

Expected: a `BHP: #>` prompt in Terminal 2. Type `whoami`, `ls`, `id`.

6.2 Test 2 — Send Text

```
# Terminal 1
python3 netcat.py -t 127.0.0.1 -p 5555 -l -c

# Terminal 2
echo 'hello world' | python3 netcat.py -t 127.0.0.1 -p 5555
```

Expected: `hello world` appears in Terminal 1.

6.3 Test 3 — Execute Single Command

```
# Terminal 1
python3 netcat.py -t 127.0.0.1 -p 5555 -l -e "id"

# Terminal 2
python3 netcat.py -t 127.0.0.1 -p 5555
```

Expected: output of `id` on the server is sent back to Terminal 2.

6.4 Test 4 — File Upload

```
# Terminal 1
python3 netcat.py -t 127.0.0.1 -p 5555 -l -u out.txt

# Terminal 2
cat anyfile.txt | python3 netcat.py -t 127.0.0.1 -p 5555
```

Expected: `out.txt` created on the server. Server prints `Saved file out.txt`.

7 Key Bugs Fixed from the Book

Bug	Description
Bug 1	Class defined after use — `NetCat` was instantiated before its definition. Python executes top-to-bottom.
Bug 2	`try` block misindented in `send()` — nested inside `if self.buffer:`, so the receive loop never ran when buffer was empty.
Bug 3	Missing comma in `subprocess.check_output()` call before `stderr=` argument.
Bug 4	Broken string literals in `argparse` epilog — split mid-word due to book's line wrapping.

8 Closing Note

This netcat clone is a learning artifact. In a real engagement you would use the actual netcat binary, `pwncat-cs`, or `chisel` depending on the task. The value of building this is not the tool itself — it is the understanding it gives you of what every tool in your toolkit is doing at the socket level.

Once this clicks, concepts like reverse shells, bind shells, port forwarding, and pivoting all become much more intuitive because you understand the underlying TCP mechanics.

> **Next steps:** Try testing this across your Kali and Metasploitable VMs. Replace `127.0.0.1` with the Metasploitable IP and practice catching shells from the server side.